



Software Defined Network Traffic Classification for QoS Optimization Using Machine Learning

Rehab H. Serag^{1,2} · Mohamed S. Abdalzaher^{3,4} ·
Hussein Abd El Atty Elsayed⁵ · M. Sobh¹

Received: 4 September 2024 / Revised: 16 January 2025 / Accepted: 27 January 2025 /
Published online: 26 February 2025
© The Author(s) 2025

Abstract

In the era of rapidly expanding network infrastructures, ensuring optimal performance and quality of service (QoS) for diverse applications face significant challenges. Traditional traffic classification (TC) methods often fall short due to their inability to adapt to the dynamic and complex nature of modern network environments. To address this limitation, this paper proposes integrating software defined network (SDN) architecture with machine learning (ML) technology. The study examined four scenarios: multiclass classification and binary classification, both before and after scaling. We used various ML models, including linear, non-linear, and hybrid models. To evaluate the performance of these models, we utilized several evaluation metrics, such as accuracy, F1 score, kappa score, ROC curve, and confusion matrix. The paper examined different feature scaling methods, including standard scaling, min-max scaling, max-abs scaling, and robust scaling. The results showed that both min-max and max-abs scaling provided the best performance enhancement across the four scaling methods. Finally, XGBoost model provided the highest performance across all scenarios, with accuracy reaching up to 99.97%.

Keywords Traffic classification · Software defined networking · Machine learning · Quality of service

1 Introduction

Managing large, multi-vendor networks with diverse technologies is becoming more expensive due to resource shortages and increasing real estate costs for service providers [1]. A new network paradigm is needed to integrate network provisioning and management across multiple domains [2].

Software defined network (SDN) offers a solution by separating the control plane from the data plane in network devices such as switches and routers [3, 4].

Traditionally, these two planes are closely connected in conventional networks, which can make managing and scaling the network more difficult [5, 6]. SDN architecture uses a central controller to manage the network [7], communicating with switches and routers via a standard protocol such as OpenFlow. OpenFlow, developed through a collaboration between Stanford University and the University of California, Berkeley, has become a widely used protocol in SDN. It was later standardized by the Open Networking Foundation (ONF) to ensure consistency and interoperability within SDN environments [8]. One of the key benefits of SDN is that it makes networks more scalable and flexible, allowing for the creation of virtual networks customized for specific applications or types of traffic [9].

Adding intelligence to networking devices is essential for simplifying optimization, organization, maintenance, and management. However, the distributed nature of networking systems complicates the inclusion of artificial intelligence (AI) for device control [10]. A key advantage of SDN is its ability to support network programmability, making integrating AI into communication networks easy [11].

Machine learning (ML) plays a critical role in facilitating AI [12] by proficiently predicting and scheduling network resources using available data inputs [13, 14]. It strives to empower computers to improve their performance autonomously over time, without the need for explicit programming [15]. ML algorithms encompass supervised, unsupervised, semi-supervised, or reinforcement learning (SL, USL, SSL, or RL respectively) approaches, which vary based on the type of data employed for model training [16, 17].

SL involves training the model with labeled data, where the correct output is known for each input. USL entails identifying patterns and relationships within unlabeled data. SSL combines elements of both SL and USL [18]. In RL, an agent learns to interact with its environment in such a way that it maximizes the rewards received [19]. Ensemble learning (EL) methods combine the strengths of several models to improve prediction accuracy and decision-making [20]. By integrating ML into SDN, network managers can create networks that are more efficient, flexible, and safe. In our paper, we focus on SL and EL ML models.

In SDN, many tasks can benefit from leveraging ML algorithms, such as traffic classification (TC) [21, 22]. Many applications now require strict QoS to meet their demands [23]. ML can classify network traffic based on the type of application or user behavior, enabling the prioritization of high-priority traffic and ensuring that critical applications receive the necessary Quality of Service (QoS) levels. This improves customer satisfaction and experience, particularly for applications requiring real-time responses, low latency, or high throughput [24].

1.1 Motivation

1.1.1 Supervised and Ensemble Learning Models in SDN

Authors in Ref. [25] explore the integration of SDN and ML for effective network TC. The study employed supervised ML models (e.g., Decision Tree (DT), and Random Forest (RF)), an ensemble model (AdaBoost), and unsupervised K-means clustering on traffic types such as DNS, Telnet, Ping, and Voice. Using Mininet for SDN simulation and the Distributed Internet Traffic Generator (D-ITG) for dataset creation, the DT model achieved the highest classification accuracy (99.81%), and the AdaBoost ensemble method achieved 99.7%.

Three ensemble-based network traffic classification algorithms (XGBoost, GBM, and LightGBM) along with SL algorithms like DT and RF, were evaluated on a real-world network traffic dataset (ISCX dataset) in Ref. [26]. The dataset was used for training and testing, with traffic classified into different classes according to QoS requirements. The results showed accuracy, ranging from 92.64 to 96.15%.

Ref. [27] has demonstrated that ensemble algorithms can serve as effective pre-processing tools for SDN, enabling robust and reliable classification across various traffic types. Performance tests were performed using traffic trace data collected with WireShark for five types of application, each representing a distinct type of traffic. The study used the XGBoost algorithm, a well-known boosted tree learning model, to predict the final results, achieving a performance of up to 99.2%.

Another effort proposed a model that integrates ML algorithms into SDN for TC [10]. This study utilized SL algorithms, including SVM, NB, and Nearest Centroid, using a custom dataset. The results demonstrated that these SL models achieved an accuracy of over The accuracy obtained for SVM is 92.3, 96.79, and 91.02% respectively.

A study focused on classifying traffic in SDNs in an application-aware manner using SL and ensemble methods, including RF, stochastic gradient boost (SGB), and XGBoost [28]. The researchers utilized a labeled dataset generated from their custom network topology and developed an SDN application specifically designed to collect OpenFlow statistics from controlled switches. The experimental results demonstrated varying levels of accuracy for the models: RF achieved an accuracy range of 73.6% to 96.0%, SGB achieved 71.2% to 93.6%, and XGBoost achieved 73.6 to 95.2%. These findings highlight the potential of ensemble learning methods in improving traffic classification performance in SDNs.

To sum up the gaps, feature scaling plays a crucial role in enhancing the accuracy and efficiency of ML models. Although some studies incorporate feature scaling in their methodologies, they often limit their analysis to a single technique. This approach overlooks the fact that different scaling techniques can influence the performance of ML models. The evaluation of multiple scaling methods is essential to determine the most effective approach for each model, ensuring optimal classification accuracy and improved performance.

1.1.2 Reinforcement Learning in SDN

The massive volume of data generated by modern applications and services often leads to network congestion, negatively impacting QoS and the overall user experience. To address this issue, the authors in Ref. [29] propose a novel congestion control approach in SDN using a Multi-Agent RL (MARL) framework. Their study evaluates the convergence of two popular RL algorithms, Q-learning and SARSA, to determine the most suitable algorithm for their proposed system. Experimental results demonstrate that Q-learning outperforms SARSA, making it the optimal learning algorithm for managing network congestion effectively.

To improve traffic management in a multi-layer hierarchical SDN [30], introduced the use of the SARSA algorithm for QoS provisioning. When a new flow arrives, the switch communicates with the SDN controller. The controller uses the SARSA algorithm to identify the QoS requirements of the flow and calculate the best traffic path to meet those requirements. Each step in the path is determined by the next hop in the switch, ensuring the flow moves efficiently from the source to the destination.

However, we plan to compare our results with studies closely aligned with our research focus, aiming to achieve and demonstrate outstanding performance.

1.1.3 Hybrid Machine Learning in SDN

In Ref. [31], the authors explored the detection and classification of conflicting flows in SDNs using various ML algorithms, such as decision tree (DT), SVM, extremely fast decision tree (EFDT), and a hybrid DT-SVM model. The EFDT and hybrid DT-SVM algorithms were specifically developed to improve performance by combining features of DT and SVM. The experiments were carried out on two different network topologies, a simple tree, and a fat tree, with flow volumes ranging from 1,000 to 100,000. The findings showed that the EFDT algorithm achieved the highest accuracy.

However, the paper does not address QoS in SDNs, leaving a gap in understanding how the hybrid model performs in scenarios requiring QoS management and optimization.

1.2 Contribution

The paper's contributions include the following:

- Applying a wider variety of ML models, including DT, SVM, KNN, XGBoost, and Hybrid models (e.g., DT-SVM, KNN-SVM).
- Evaluating the performance of these models using multiple metrics to provide a comprehensive analysis and showing that the models' performance is consistent across various train-test split ratios.

- Implementing different feature scaling methods and pointing to the one that achieves the highest accuracy for each model.
- Analyzing the impact of feature scaling on the performance of the designated ML models.
- Investigating the computational efficiency of each model, highlighting the trade-offs between accuracy and execution time.

The paper is organized as follows: Sect. 2 provides the methodology. ML models used for TC are discussed in Sect. 3, while Sect. 4 presents the used evaluation metrics. Results and discussion are introduced in Sect. 5. Finally, Sect. 6 covers the conclusion and future work.

2 Methodology

This section describes the approach used to address the research problem in which the IEEE DataPort dataset is used. We preprocessed the data to address class imbalance and evaluate ML models under various scenarios.

2.1 Dataset

The dataset used in this study was obtained from IEEE DataPort [32]. The dataset is exclusively derived from OpenFlow switch statistics and processed by an SDN application. It includes 37 features, each record capturing information periodically collected from flows within an OpenFlow switch. In total, the dataset contains 662,828 records, consisting of 221,564 normal records and 441,264 attack records. The parameters are grouped as follows:

- Flow Identifier Features: Include parameters like flow-id, EtherType (eth-type), source ipv4 address (ipv4-src), destination ipv4 address (ipv4-dst), IP protocol (ip-proto), source port number (src-port), destination port number (dst-port), and flow-duration for flow identification (flow-duration).
- Packet-Based Features: Cover statistics such as total packets per flow, packets per time unit, and sample periods.
- Byte-Based Features: Provide information on total bytes per flow, bytes per flow per time unit, and sample periods.
- Flow-Timer Features: Track uptime and downtime for each flow.
- Cumulative Features: Aggregate metrics like total packets/bytes per source and destination IP, and total connections per source and destination IP, offering an overview of network activity.

2.2 Dataset Preprocessing

In our study, we focus on the normal records dataset, excluding the attack records. The dataset's Category feature consists of 7 applications: Streaming, HTTP, DNS,

FTP, SSH, ICMP, and NTP. The number of records for each application varies significantly, which can negatively impact the performance of ML models by creating biased models that favor the majority classes, thus reducing overall accuracy and reliability.

To address this issue, we balanced the dataset by removing records from over-represented categories to ensure an equal number of records for each application. Our approach is evaluated across four scenarios:

- Scenario 1: Multi-class classification using the above mentioned seven application labels without scaling the features.
- Scenario 2: Multi-class classification using the above mentioned seven application labels with scaling the features.
- Scenario 3: Binary classification with two labels without scaling the features.
- Scenario 4: Binary classification with two labels with scaling the features.

For Scenarios 1 and 2, each application was then assigned a numerical label from 0 to 6, serving as the target variable for our classification models. In Scenarios 3 and 4, we assigned two labels: streaming (label "0") and all other applications (label "1"), to evaluate how our models can prioritize streaming traffic, which is an important factor in ensuring QoS.

In the binary classification scenarios (Scenarios 3 and 4), we encountered an imbalance where streaming records were significantly fewer than the combined records of other applications. To balance the data, we applied an augmentation algorithm, the synthetic minority over-sampling technique (SMOTE) [33, 34], generating additional synthetic streaming records to create a more balanced dataset.

It is worth mentioning that scaling techniques play a crucial role in optimizing the performance of ML models, so it is used in scenarios 2 and 4. In this analysis, we compare the effectiveness of different scaling methods-standard scaling, min-max scaling, maxabs scaling, and robust scaling-on various ML models, including SVM, DT, KNN, XGBoost, Hybrid DT-SVM, and Hybrid SVM-KNN. The results are presented in two tables, one for multi-class classification and the other for binary classification. Figure 1 summarizes the workflow of the proposed approach including the dataset preprocessing.

2.3 Integration of ML Model in SDN Architecture

As shown in Fig. 2, the ML model would be integrated into the SDN control plane. The control plane is responsible for making decisions about how traffic should be handled across the network, which includes TC and QoS optimization. When a packet arrives at a switch in the data plane, it is initially checked for an existing flow entry. If no matching rule is found, the packet is sent to the control plane using a Packet-In message. The SDN controller then processes the packet and uses the ML model to classify the traffic, determining its type and specific QoS requirements. Based on the classification results, the controller calculates the necessary QoS policies and sends the information back to the switch in the form of a Flow-Mod

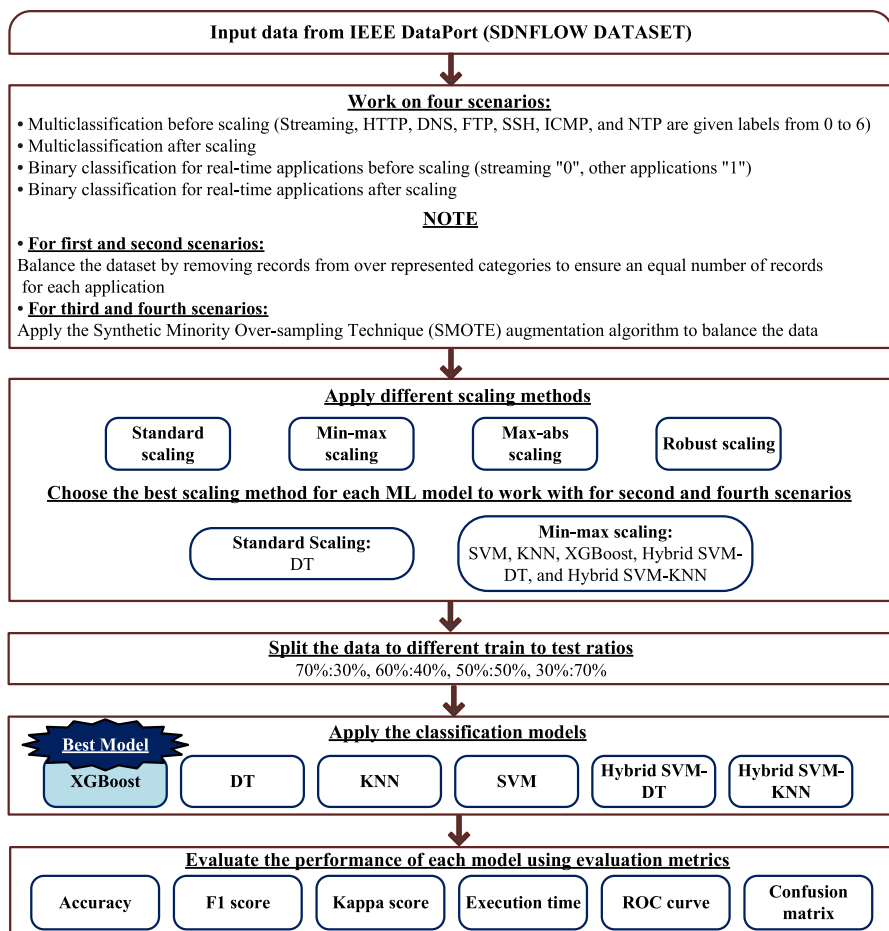


Fig. 1 Overview of the data preprocessing and classification process used in the SDNFLOW dataset study

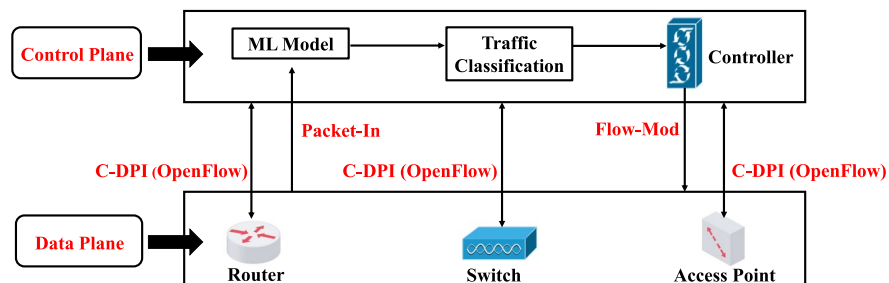


Fig. 2 Integration of machine learning models in SDN architecture

message. The switch then updates its flow table with the new rules, enabling it to forward the packet according to the newly defined QoS policies.

3 ML Models Used for Traffic Classification

ML for TC is a powerful technique that enables computers to automatically learn patterns and make decisions based on labeled data. By applying algorithms and statistical models, ML classifiers categorize input data into predefined classes or labels. This process involves training the model on a labeled dataset to learn how input features relate to target labels. Once trained, the classifier can generalize to new, unseen data, accurately predicting the class labels of future inputs.

To implement the proposed solution in an SDN environment, the process is straightforward as it mainly involves calling predefined functions. The SDN controller, such as Ryu or OpenDaylight, manages traffic by collecting flow statistics from OpenFlow-enabled switches, which are then used as features for TC model. The XGBoost model (highest accuracy), is integrated for real-time classification with minimal configuration, prioritizing latency-sensitive traffic and adjusting resources for less critical traffic. Tools like Mininet can be easily installed with just a few commands and are commonly used to simulate SDN networks without complex setup. Additionally, Wireshark can be used to monitor and validate performance, with little configuration required.

Here, we outline a classification of the prevalent linear and nonlinear ML models utilized in our framework.

3.1 Linear Classification Models

3.1.1 Support Vector Machine (SVM) with Linear Kernel

Linear SVMs are a subtype of SVMs that are specifically designed for linearly separable data or data that can be transformed into a linearly separable form in the feature space. These models aim to find a hyperplane that best separates the classes in the input data with the maximum margin, which is the distance between the hyperplane and the closest data points (support vectors) from each class [35, 36].

3.2 Non-Linear Classification Models

3.2.1 Decision Tree (DT)

This method is displayed as a tree structure, where the features of the dataset are represented by internal nodes, and the branches correspond to the decision criteria, with outcomes expressed by leaf nodes [31]. This approach determines information gains by calculating entropy on the dataset, which is then used to classify the data. The entropy H of a dataset is defined as:

$$H(D) = - \sum_{k=1}^n p_k \log_2 p_k, \quad (1)$$

where p_k is the proportion of instances belonging to class k in the dataset D . Using this entropy calculation, the algorithm identifies the root node based on the highest information gain. This process is repeated to split branches and complete the tree [37].

DTs have several advantages, such as being easy to interpret and visualize, automatically performing feature selection, and effectively handling non-linear relationships between parameters. One major benefit of DTs is that they do not require feature scaling. This is because DTs work by splitting data based on feature thresholds, rather than relying on distance calculations or gradients. This characteristic makes DTs especially robust when working with datasets that include features with different scales or units. As a result, DTs are a straightforward yet powerful tool for both classification and regression tasks, making them a popular choice in various machine learning applications.

3.2.2 K-Nearest Neighbors (KNN)

KNN is another supervised learning technique that classifies an unclassified sample by looking at the classes of its k nearest neighbors [38]. The KNN algorithm is based on a simple idea: If most of a sample's K nearest neighbors belong to a particular class, the sample is classified as part of that class [39]. The algorithm follows these steps:

1. Select the value of K , which indicates the number of neighbors to consider.
2. Compute the Euclidean distance for each of the K neighbors.
3. Identify the K nearest neighbors based on these distances.
4. Count how many data points from each class are among the K neighbors.
5. Assign the new data point to the class that appears most frequently among the K neighbors.

KNN is easy to implement, achieves high accuracy, computes features efficiently, and is suitable for multiclass classification. However, KNN can be time-consuming when dealing with large datasets [40].

Feature scaling significantly impacts the performance of KNN. Since it relies on distance calculations to identify the nearest neighbors, features with larger scales can disproportionately influence the distance metric, leading to biased classifications [41].

3.2.3 Extreme Gradient Boosting (XGBoost)

XGBoost, short for "Extreme Gradient Boosting," is a ML algorithm that implements the gradient boosting framework [42]. It is designed to be efficient and flexible [43]. Gradient boosting is an ensemble ML technique aimed at enhancing the effectiveness

of weak learners, such as classifiers or regressors, transforming them into robust learners [44]. The method operates by iteratively training multiple models, with each subsequent model concentrating on the examples that previous models misclassified. This sequential approach assigns greater importance to misclassified instances, enabling subsequent models to focus more on correcting these errors. As a result, individual predictors become specialized in different segments of the dataset, leading to a reduction in both bias and variance [45].

3.3 The Proposed Hybrid Classification Models

3.3.1 SVM-DT

The Hybrid SVM-DT model combines the strengths of both SVMs and DTs to improve classification performance. This hybrid approach aims to utilize SVM's ability to optimize the margin between classes and DT's efficiency in decision-making, providing a balanced and effective classification method.

3.3.2 KNN-SVM

The Hybrid SVM-KNN model merges the capabilities of SVM and KNN to create a robust and flexible classifier. SVM performs exceptionally well at finding the optimal hyperplane that separates different classes, which is especially useful in high-dimensional datasets. Meanwhile, KNN is a straightforward algorithm that classifies data points based on the majority class among their closest neighbors. By combining these methods, the Hybrid SVM-KNN model can benefit from SVM's margin optimization and KNN's local, instance-based learning approach, resulting in a powerful tool for classification tasks.

4 Evaluation Metrics

This section outlines the metrics used to evaluate the performance of ML models. These will help measure the effectiveness and reliability of the developed approaches in different scenarios.

4.1 Accuracy

Accuracy is a widely used metric for evaluating classification models. It shows how often the model's predictions match the actual outcomes. Specifically, accuracy is calculated by dividing the number of correct predictions by the total number of predictions. The formula for accuracy is given by:

$$Accuracy = \frac{P_C}{P_T}, \quad (2)$$

where P_C represents the count of correct predictions and P_T is the total count of predictions. Accuracy works well when the classes in the dataset are balanced, meaning each class has roughly the same number of instances. However, it can be misleading with imbalanced datasets, where one class is much more common than others. For example, in a dataset where one class dominates, a model that always predicts the majority class will have high accuracy, even though it might not perform well on the minority class [46].

Despite this, accuracy remains a popular metric for evaluating overall model performance. To get a clearer picture, it is often used with other metrics like precision, recall, and F1 score.

4.2 F1-Score

The F1 score is a metric that helps in the evaluation of a classification model by balancing both precision and recall [47]. It is especially useful when dealing with imbalanced classes, where some classes are much more common than others. Unlike accuracy, which might give a false sense of how well the model is performing when classes are imbalanced. By considering both precision and recall, the F1 score provides a clearer picture of a model's performance in challenging scenarios. The F1 score is defined by the formula:

$$F1score = 2 \times \frac{Pr \times R}{Pr + R}, \quad (3)$$

where Pr is the precision and R is the recall. The F1 Score ranges from 0 to 1, where 1 represents perfect precision and recall, and 0 indicates the poorest performance.

4.3 Kappa Score

The Kappa score, or Cohen's Kappa, is a metric that evaluates how well a classification model performs by accounting for the accuracy that might occur just by chance. The formula for Cohen's Kappa is:

$$\kappa = \frac{A_o - A_e}{1 - A_e}, \quad (4)$$

where A_o is the observed accuracy and A_e is the expected accuracy. The Kappa score ranges from -1 to 1 . A score of 1 means perfect agreement between the model's predictions and the actual outcomes. A score of 0 means the model's performance is as effective as random chance. Negative values indicate that the model performs worse than what you'd expect by random guessing.

4.4 ROC Curve

The Receiver Operating Characteristic (ROC) curve is a tool for evaluating the performance of classification models. It graphs the true positive rate (TPR) against the

false positive rate (FPR) across different thresholds. The TPR, also called sensitivity or recall, measures how well the model correctly identifies positive instances. On the other hand, the FPR shows the ratio of false positives (incorrectly predicted positives) to all actual negative instances.. It is calculated as:

$$FPR = \frac{FP}{FP + TN}. \quad (5)$$

The ROC curve is created by varying the decision threshold of the classifier. Each point on the ROC curve represents a TPR and FPR pair corresponding to a specific threshold.

The AUC of ROC curve is a single scalar value that summarizes the overall performance of a classifier. A perfect model has an AUC of one. A model with a high TPR and low FPR will have a curve that bows towards the top-left corner of the plot.

4.5 Confusion Matrix

A confusion matrix is a tool used to assess the performance of the classification model by providing a detailed breakdown of correct and incorrect predictions. The matrix compares the actual target values with the values predicted by the model. The confusion matrix is extended to an $K \times K$ matrix, where K is the number of classes. Each cell in the matrix represents the count of instances for a true class (row) and a predicted class (column). Moreover, the confusion matrix enables the calculation of various evaluation metrics such as precision, recall, and F1 score.

5 Results and Discussion

Our experiment results are shared in this section. We analyze how each model performs under different conditions in the designated SDN environment. Different evaluation metrics are used to assess their effectiveness and stability, and we discuss what these results mean in real world terms, as well as highlight each model's strengths and weaknesses.

The modeling of the proposed solution was conducted using Anaconda Jupyter Notebook as the development environment, with key Python libraries including Pandas for data manipulation, Scikit-learn for machine learning model training and evaluation, Matplotlib for visualizing results such as ROC curves, and Seaborn for enhanced visualization of the confusion matrix. The experiments were performed on a laptop with an AMD Ryzen 7 7730U processor (2.00 GHz), 16 GB RAM, and a 64-bit Windows operating system.

5.1 Evaluation of Different Train-Test Split Ratios

We examined four different train-test split ratios of the utilized SDN dataset as shown in Tables 1 and 2 to evaluate the performance of our models. Since the results are consistent across different test values, we will present only the results for test =

Table 1 Accuracy percentage of different test values in multi-class classification scenario before scaling

Test values	SVM	DT	KNN	XGBoost	Hybrid KNN-SVM	Hybrid DT-SVM
Test = 30%	41.67	99.81	61.22	99.95	60.75	99.81
Test = 40%	42.19	99.86	61.04	99.96	60.72	99.86
Test = 50%	41.62	99.86	61.11	99.91	60.14	99.86
Test = 70%	41.42	99.55	58.18	99.92	57.75	99.7

Table 2 Accuracy percentage of different test values in multi-class classification scenario after scaling

Test values	SVM	DT	KNN	XGBoost	Hybrid KNN-SVM	Hybrid DT-SVM
Test = 30%	99.91	99.81	99.53	99.95	99.76	99.81
Test = 40%	99.86	99.86	99.6	99.96	99.79	99.86
Test = 50%	99.83	99.72	99.52	99.91	99.8	99.74
Test = 70%	99.47	99.61	99.09	99.91	99.61	99.43

Table 3 Comparison of model performance (accuracy percentage) using various scaling techniques for multi-class classification

ML models	Standard scaling	Min-Max scaling	MaxAbs scaling	Robust scaling
SVM	84.94	99.91	99.91	30.49
DT	99.81	99.76	99.81	99.81
KNN	78.125	99.53	99.53	92.66
XGBoost	99.95	99.95	99.95	99.95
Hybrid DT-SVM	99.76	99.81	99.81	99.81
Hybrid SVM-KNN	85.41	99.76	99.72	92.09

30% in our paper. The test split ratio of 30% is chosen for presentation because it provides a balanced representation of model performance while maintaining consistency across various test ratios. Our analysis shows that the results are stable and similar across different test ratios, indicating that a 30% test split is sufficiently representative of the models' general performance. This allows us to clearly illustrate the evaluation metrics without redundancy.

5.2 Impact of Scaling Techniques on Model Performance

The comparison of model performance with various scaling techniques for multi-class and binary-class classification as presented in Tables 3 and 4 reveals notable differences. In the multi-class classification, SVM shows substantial improvement with min-max and maxabs scaling, achieving almost perfect performance, while with robust scaling it drops significantly to 30.49%. DT and XGBoost maintain consistently high accuracy across all scaling methods, with slight variations. KNN and

Table 4 Comparison of model performance (accuracy percentage) using various scaling techniques for binary classification

ML models	Standard scaling	Min-Max scaling	MaxAbs scaling	Robust scaling
SVM	91.45	99.97	99.97	56.90
DT	99.97	99.97	99.97	99.97
KNN	87.18	99.73	99.73	97.39
XGBoost	99.97	99.97	99.97	99.97
Hybrid DT-SVM	99.97	99.97	99.97	99.97
Hybrid SVM-KNN	92.26	99.97	99.97	97.39

Hybrid SVM-KNN models also benefit markedly from min-max and maxabs scaling, with a noticeable performance drop when robust scaling is applied. In binary classification, SVM again shows excellent performance with min-max and maxabs scaling, achieving 99.97%, but its performance deteriorates with robust scaling. DT and XGBoost exhibit uniformly high performance across all scaling techniques. KNN and Hybrid SVM-KNN models perform exceptionally well with min-max and maxabs scaling and retain high accuracy with robust scaling. Overall, min-max and maxabs scaling emerge as the most effective techniques for enhancing model performance, particularly for SVM, KNN, and hybrid models; where the accuracy ranges from 99.53% to 99.97%. These findings underscore the importance of selecting appropriate scaling techniques to optimize model performance, with min-max and maxabs scaling being the most reliable choices across different models and classification tasks.

For each model, we used the optimal scaling method identified in our initial analysis to evaluate our scenarios. The scenarios are evaluated using five key metrics: accuracy, F1 score, Kappa score, ROC curve, and confusion matrix. These metrics provide a comprehensive understanding of the models' performance from various perspectives.

5.3 Models Performance Using Accuracy, F1-Score, and Kappa Score

We will start by discussing scenarios 1 and 3, which are without scaling. Then, scenarios 2 and 4 after data scaling will be illustrated to emphasize the ML models accuracy improvement after scaling.

For scenario 1: the DT model as shown in Figs. 3 and 4 demonstrates high accuracy (99.81%), F1 score (99.80%), and Kappa score (99.75%), indicating that it effectively captures the structure of the dataset. The high performance of DT can be attributed to its ability to handle complex decision boundaries and its robustness to overfitting with proper pruning. The execution time is also low (3.35 s), making DT a practical choice for efficient and accurate classification.

In contrast, the SVM model performed poorly with low accuracy (41.67%), F1 score (38.18%), and Kappa score (19.49%), underscoring its reliance on feature scaling for optimal performance. Its high execution time (97.62 s) further highlights the

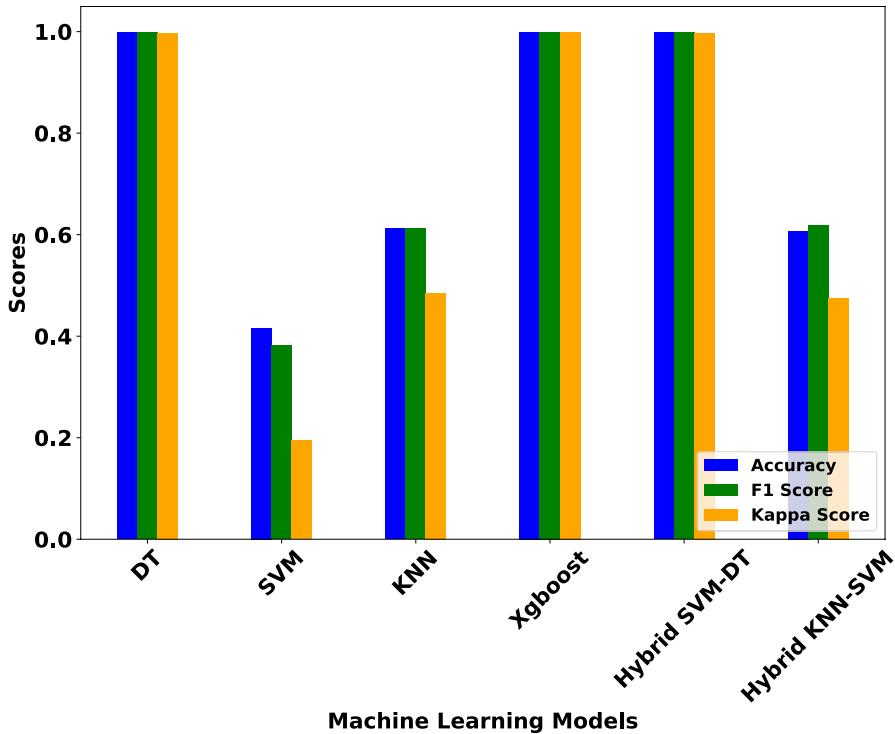


Fig. 3 Performance comparison of ML models for multi-class classification before scaling

computational challenges of using SVM on unscaled data, emphasizing the importance of preprocessing steps like normalization.

KNN achieved moderate performance with an accuracy of 61.22%, F1-score 61.22%, and Kappa score 48.49%. These results reflect its sensitivity to unscaled features, which affect distance-based computations. Despite this, KNN took the least time to run (1.66 s) making it computationally efficient.

XGBoost emerged as one of the best-performing models, with accuracy (99.95%), F1 score (99.95%), and Kappa score (99.94%). Its success is due to the boosting approach, which combines weak learners into a strong classifier. Furthermore, its low execution time (2.81 s) makes it well-suited for high-performance applications.

The hybrid SVM-DT model achieved metrics comparable to the standalone DT model, with accuracy (99.81%), F1 score (99.81%), and Kappa score (99.75%), reflecting the dominant influence of the DT component. However, its high execution time (96.66 s) points to the computational cost introduced by the SVM component. This model demonstrates both the advantages of hybrid approaches in maintaining high accuracy and their disadvantages in terms of computations.

The results indicate that the hybrid KNN-SVM approach has almost the same level of accuracy as the standalone KNN method with moderate accuracy, F1 score, and Kappa score (60.75%, 61.76%, and 47.56% respectively). This similarity suggests that KNN's characteristics dominate the hybrid model's performance. The

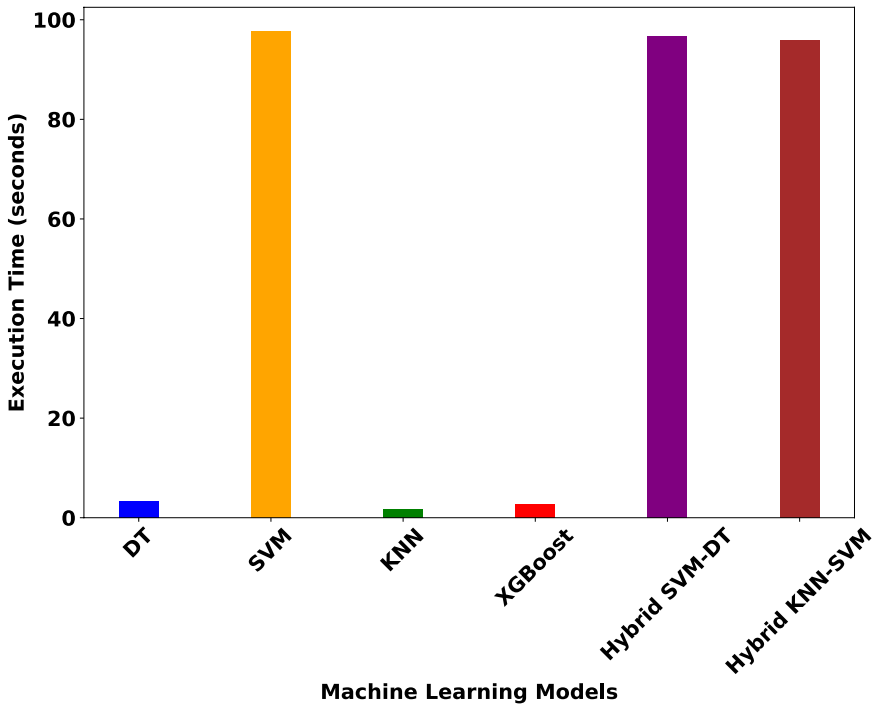


Fig. 4 Execution time comparison of ML models for multi-class classification before scaling

high execution time (95.89 s) reflects the added complexity of combining the two methods.

For scenario 3: as observed in Figs. 5 and 6, DT demonstrated near-perfect performance, with accuracy (99.97%), F1 score (99.97%), and Kappa score (99.94%). Its low execution time (1.8 s) further reinforces its efficiency.

SVM underperformed in comparison to DT, with lower accuracy (79.05%), F1 score (78.21%), and Kappa score (57.87%). Its high execution time (146.61 s) limits its practicality without further optimization.

KNN achieved high accuracy (94.96%), F1 score (94.96%), and Kappa score (89.92%) though slightly lower than DT. Despite this, its execution time is moderate (4.07 s), balancing between computational efficiency and performance.

XGBoost matched DT's performance metrics and maintained a low execution time (3.11 s), showcasing its robustness and efficiency.

The hybrid SVM-DT model matched the individual DT model's performance but exhibited a higher execution time (135.55 s), while the hybrid KNN-SVM model showed slightly improved metrics over standalone KNN (96%, 96%, and 92%, respectively), with a significant increase in execution time (139.27 s).

For scenario 2: as presented in Figs. 7 and 8, post-scaling, SVM shows a marked improvement in accuracy, F1 score, and Kappa score (99.91, 99.89, and

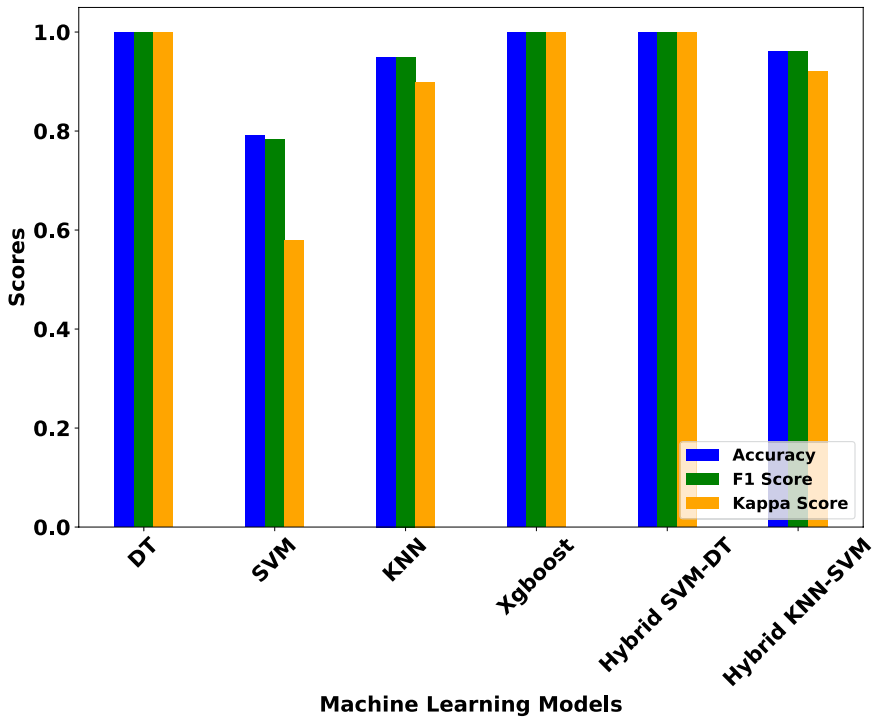


Fig. 5 Performance comparison of ML models for binary classification before scaling

99.88% respectively). Despite a reduced execution time (19.63 s), it remains computationally intensive.

DT maintained its high metrics: accuracy (99.81%), F1 score (99.81%), and Kappa score (99.75%). Its ability to handle scaled data efficiently is supported by its low execution time (1.6 s).

The KNN model also benefitted from scaling, with accuracy (99.53%), F1 score (99.51%), and Kappa score (99.58%). Feature normalization enhanced its performance while retaining a low execution time (3.34 s).

XGBoost continued its strong performance with accuracy (99.95%), F1 score (99.95%), and Kappa score (99.94%). Its execution time (2.21 s) remained low, reinforcing its suitability for computationally efficient, high-performance tasks.

The hybrid SVM-DT model achieved comparable metrics to DT alone but with a higher execution time (18.68 s), reflecting the SVM component's complexity. Similarly, the hybrid KNN-SVM model improved post-scaling, with accuracy (99.76%), F1 score (99.76%), and Kappa score (99.69%). However, its high execution time (21.4 s) highlights the computational trade-offs of hybrid approaches.

For scenario 4: as shown in Figs. 9 and 10, DT, SVM, and XGBoost demonstrated near-identical performance metrics (99.97, 99.97, and 99.94% for accuracy, F1 score, and Kappa score, respectively). However, SVM had the longest

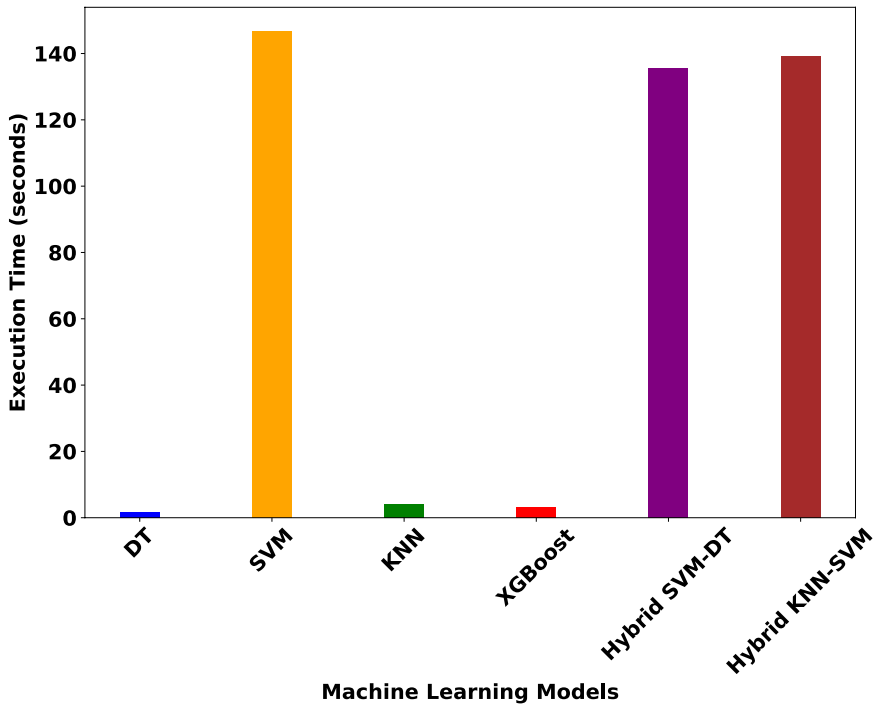


Fig. 6 Execution time comparison of ML models for binary classification before scaling

execution time, highlighting its computational cost despite its strong classification ability post-scaling.

KNN performed slightly lower, with accuracy (99.73%), F1 score (99.73%), and Kappa score (99.46%). Its moderate execution time (3.98 s) balances performance and efficiency.

The hybrid SVM-DT and KNN-SVM models maintained similar accuracy levels to their standalone components but at the cost of significantly higher execution times (13.58 s and 21.4 s, respectively).

5.4 Models Performance Using ROC Curve Results and Confusion Matrix

For scenarios 1 and 2: the final results obtained by ROC curves are mentioned in Tables 5 and 6, showing significant improvements after scaling the features. Before scaling, DT, XGBoost, and SVM-DT models achieved near-perfect classification ($AUC = 1$) for all classes except Class 5, which had an AUC of 0.85. SVM showed inconsistent performance across classes, with AUCs ranging from 0.44 to 0.84, while KNN performed well overall, though with lower AUCs for Class 3 (0.74) and

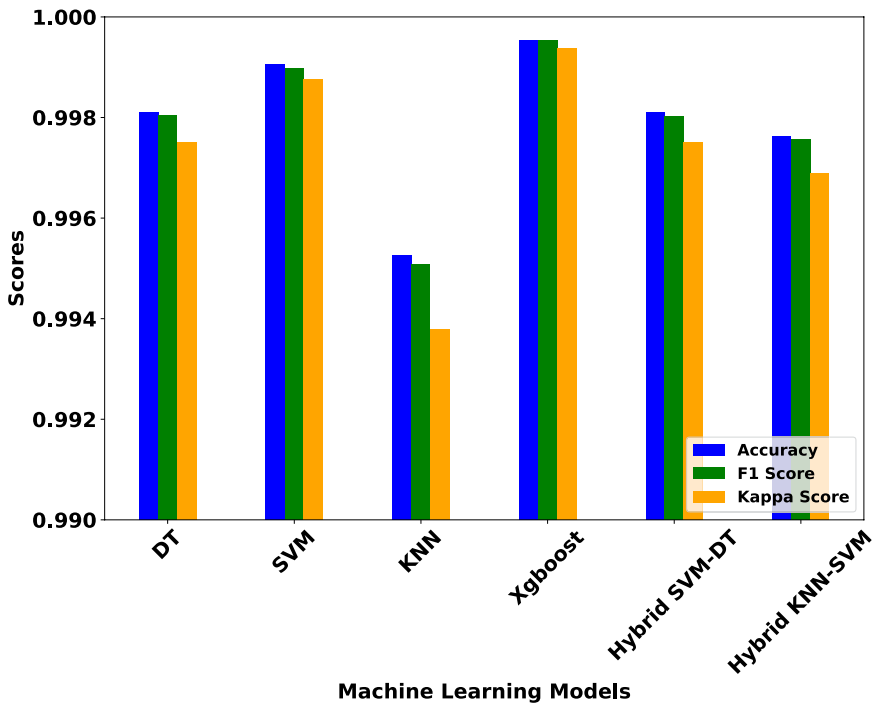


Fig. 7 Performance comparison of ML models for multi-class classification after scaling

Class 5 (0.93). The hybrid KNN-SVM model displayed robust performance, with slightly reduced performance for Class 6 (AUC = 0.88).

After scaling, all models achieved perfect classification (AUC = 1) across all classes. This demonstrates the significant positive impact of feature scaling, particularly for SVM and KNN, which rely on distance-based calculations. The reduced performance observed for Class 5 before scaling highlights sensitivity to class imbalances, as this class had significantly fewer labels than others.

For the binary classification scenarios (scenarios 3 and 4) shown in Tables 7 and 8, DT, XGBoost, and hybrid SVM-DT models achieved an AUC of 1 for both labels before scaling, indicating perfect performance. SVM showed moderate performance with an AUC of 0.86 for both labels, while KNN exhibited high performance with an AUC of 0.99. Both achieved perfect classification (AUC = 1) for both labels, and hybrid KNN-SVM displayed high performance with an AUC of 0.99.

After scaling, all models demonstrated perfect classification (AUC = 1) for both classes. This indicates that feature scaling positively impacts binary classification performance, particularly for SVM and KNN.

As shown in Fig. 11, the confusion matrices illustrate the performance of the best models for different scenarios. In Fig. 11a, the XGBoost model demonstrates excellent performance in the first and second scenarios, with nearly no misclassifications across the 7-label dataset. This indicates that XGBoost is well-suited for handling

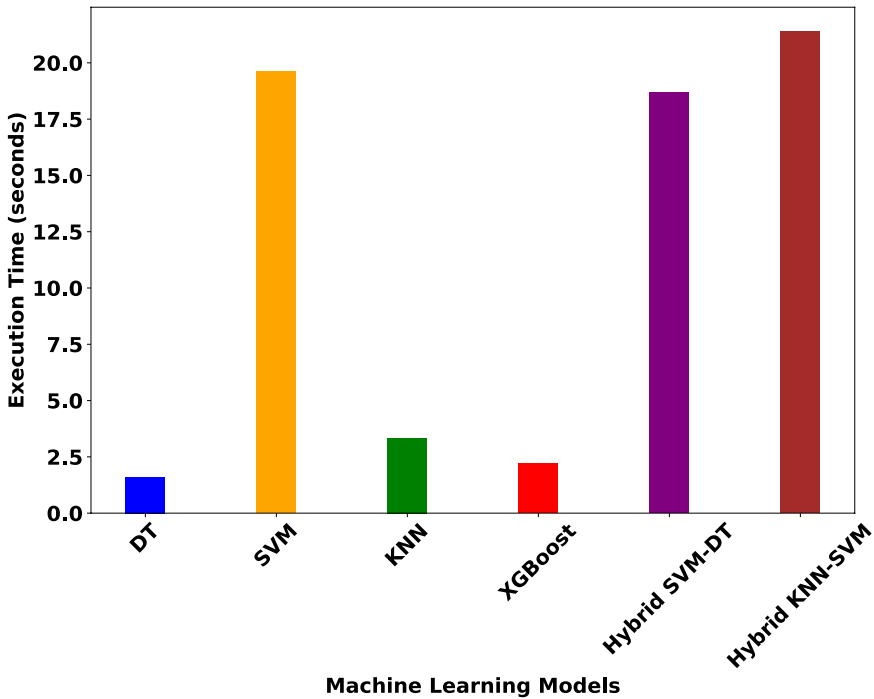


Fig. 8 Execution time comparison of ML models for multi-class classification after scaling

the complexity of multi-class data, effectively balancing between precision and recall for all classes.

In Fig. 11b, the models employed in the third scenario (XGBoost, DT, and hybrid DT-SVM) and in the fourth scenario (XGBoost, DT, SVM, hybrid DT-SVM, and hybrid KNN-SVM) exhibit strong performance. While near-perfect classification is observed in many cases, slight variations in performance across models suggest that some algorithms may be more sensitive to specific characteristics of the dataset, such as label distribution or feature scaling.

5.5 Summary of Key Findings

The following points summarize the performance of the four scenarios with and without feature scaling, evaluated using accuracy, F1-score, Kappa score, ROC curve values, and confusion matrices:

- Feature scaling had a significant positive impact on model performance, particularly for SVM and KNN, highlighting the importance of appropriate preprocessing in improving classification outcomes.

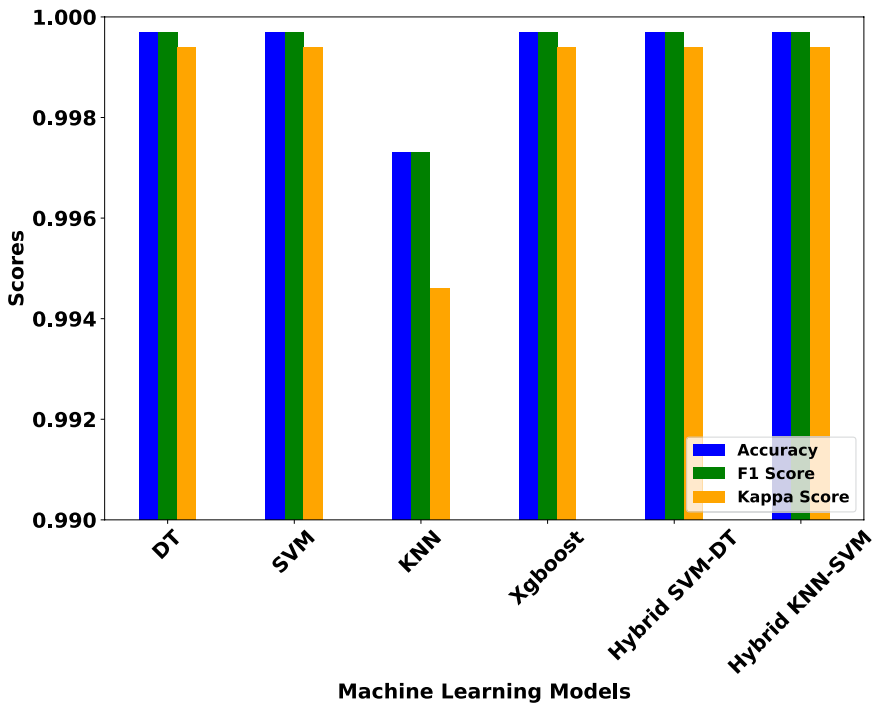


Fig. 9 Performance comparison of ML models for binary classification after scaling

- DT and XGBoost consistently delivered high accuracy and efficiency, making them well-suited for real-time applications where both performance and speed are critical.
- Hybrid models achieved high performance but at the cost of increased execution times, indicating a trade-off between robustness and computational efficiency.
- Hybrid models, while computationally intensive, also benefit from scaling, demonstrating improved performance. Therefore, for optimal model performance, especially for those models depending on distance metrics, feature scaling is a crucial preprocessing step.

Table 9 presents a comparison of the accuracy of our proposed ensemble model (XGBoost) with the EL models used in previous studies. It is clear that our model consistently outperforms the others in terms of accuracy. The results indicate that XGBoost, as the best-performing model in our approach, achieves superior classification accuracy compared to the ensemble models tested in prior research.

Despite the promising results, this study has some limitations. The dataset used is suitable for training and evaluation, but it may not fully capture the complexity of real-world network traffic, which could affect performance in dynamic settings. Although techniques like SMOTE were used to address class imbalance, more

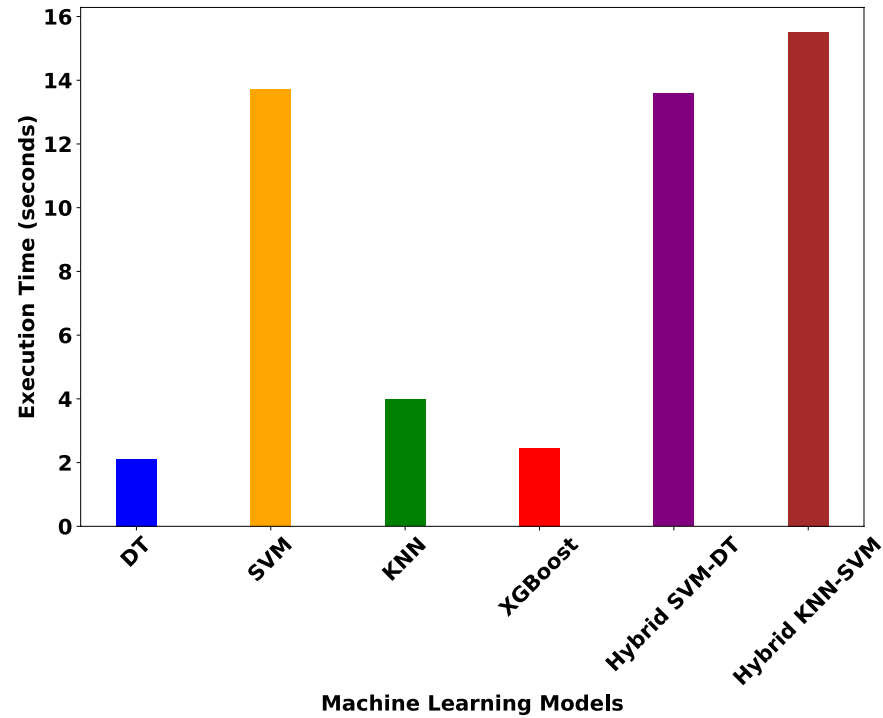


Fig. 10 Execution time comparison of ML models for binary classification after scaling

severe imbalances in practical scenarios could cause additional challenges. Additionally, models like SVM and KNN are sensitive to feature scaling, introducing preprocessing overhead. Finally, hybrid models (SVM-DT and KNN-SVM) may face scalability challenges in real-time applications due to their higher computational demands.

Table 5 ROC curve AUC values for multi-class classification before scaling

ML models	CLASS 0	CLASS 1	CLASS 2	CLASS 3	CLASS 4	CLASS 5	CLASS 6
DT	1	1	1	1	1	0.85	1
SVM	0.84	0.62	0.44	0.61	0.48	0.57	0.63
KNN	0.96	0.78	0.97	0.74	0.95	0.93	0.86
XGBoost	1	1	1	1	1	1	1
Hybrid SVM-DT	1	1	1	1	1	0.85	1
Hybrid KNN-SVM	0.97	0.78	0.99	0.75	0.97	0.91	0.88

Table 6 ROC curve AUC values for multi-class classification after scaling

ML models	CLASS 0	CLASS 1	CLASS 2	CLASS 3	CLASS 4	CLASS 5	CLASS 6
DT	1	1	1	1	1	1	1
SVM	1	1	1	1	1	1	1
KNN	1	1	1	1	1	0.93	1
XGBoost	1	1	1	1	1	1	1
Hybrid SVM-DT	1	1	1	1	1	1	1
Hybrid KNN-SVM	1	1	1	1	1	1	1

Table 7 ROC curve AUC values for binary classification before scaling

ML models	CLASS 0	CLASS 1
DT	1	1
SVM	0.86	0.86
KNN	0.99	0.99
XGBoost	1	1
Hybrid SVM-DT	1	1
Hybrid KNN-SVM	0.99	0.99

6 Conclusion and Future Work

This paper studied the integration of SDN with ML to enhance TC for QoS optimization in modern network environments. The study covered four scenarios: multi-class classification and binary classification, both before and after scaling. Various ML models, including linear, non-linear, and hybrid models, were evaluated using five metrics which are accuracy, F1 score, kappa score, ROC curve, and confusion matrix. Among the models tested, XGBoost consistently demonstrated the best performance, achieving an accuracy of up to 99.97% across all scenarios. The paper also compared between four different feature scaling methods which are min-max, max-abs, standard, and robust scaling. Min-max and max-abs scaling emerged as the most effective, providing the highest accuracy compared to standard scaling, and robust scaling. Furthermore, the results show that DT and XGBoost provide high accuracy and fast execution, making them ideal for real-time applications. These classification methods optimize QoS in SDN environments by enabling detailed

Table 8 ROC curve AUC values for binary classification after scaling

ML models	CLASS 0	CLASS 1
DT	1	1
SVM	1	1
KNN	1	1
XGBoost	1	1
Hybrid SVM-DT	1	1
Hybrid KNN-SVM	1	1

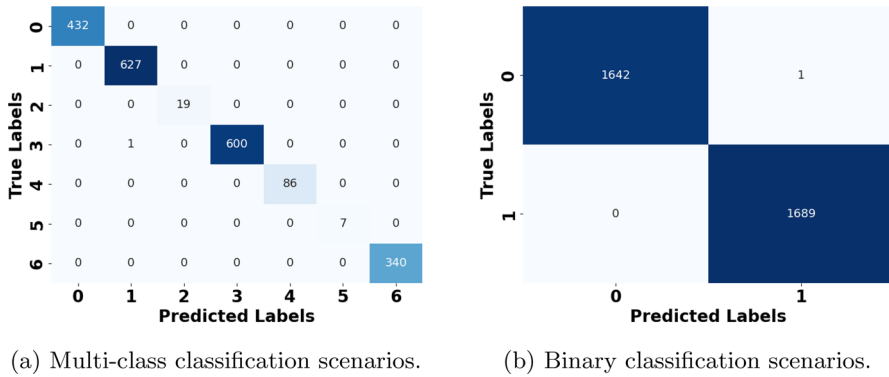


Fig. 11 Confusion matrix of the Best models in each scenario. **a** Best model performance in multi-class classification scenarios which is XGBoost, and **b** Best models for binary classification before scaling (XGBoost, DT, Hybrid DT-SVM) and after scaling (XGBoost, DT, SVM, Hybrid DT-SVM, Hybrid KNN-SVM)

traffic management with multi-class classification and simplifying prioritization with binary classification. Combining both approaches allows SDN administrators to maintain stability and responsiveness under varying network conditions.

In terms of future work, deploying these models in real-world SDN environments could provide valuable insights into their performance under dynamic conditions. It would also be beneficial to extend the work into areas like edge and fog computing, which are becoming increasingly important in modern networks. Moreover, incorporating more advanced learning methods could improve the models' scalability.

Table 9 Comparison of our ensemble learning technique accuracy (best) with previous studies

Refer-ences	Dataset	Ensemble meth-ods	Accuracy
[25]	Distributed Internet Traffic Generator (D-ITG) for dataset creation	AdaBoost	99.7
[26]	Real-world network traffic dataset (ISCX dataset)	XGBoost, GBM, and LightGBM	94.05, 94.88, and 96.15
[27]	Traffic trace data collected with Wire-Shark for five types of application	XGBoost	99.2
[28]	Dataset generated from their custom network topology	SGB and XGBoost	Range from 71.2 to 93.6%, and 73.6 to 95.2% respectively
Our study	Dataset obtained from IEEE DataPort	XGBoost	Range from 99.95 to 99.97

Acknowledgements The authors would like to thank the dataset creators for making it publicly available. Their contribution has greatly facilitated our research and enabled us to achieve our study objectives.

Author Contributions Conceptualization, M.S.A.; methodology, R. H. S., M.S.A., H.A.E., and M.S.; validation, R. H. S., M.S.A., and H.A.E.; formal analysis, R. H. S., M.S.A., H.A.E., and M.S.; investigation, R. H. S., M.S.A., and H.A.E.; resources, R. H. S. and M.S.A.; writing-original draft preparation, R. H. S., M.S.A., and H.A.E.; writing-review and editing, R. H. S., M.S.A., and H.A.E.; visualization, R. H. S., and M.S.A.; supervision, M.S.A., H.A.E., and M.S.; project administration, M.S.A. and H.A.E. All authors have reviewed the manuscript.

Funding Open access funding provided by The Science, Technology & Innovation Funding Authority (STDF) in cooperation with The Egyptian Knowledge Bank (EKB). No funding was received for conducting this study.

Data Availability The data presented in this study are available upon reasonable request.

Code Availability Not applicable.

Declarations

Conflict of interest The authors have no Conflict of interest/Conflict of interest to declare that are relevant to the content of this article.

Ethical Approval All the author declare their ethics approval.

Consent to Participate All the author declare their consent to participate.

Consent for Publication All the author declare their consent for publication.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

References

1. Awad, M.K., Ahmed, M.H.H., Almutairi, A.F., Ahmad, I.: Machine learning-based multipath routing for software defined networks. *J. Netw. Syst. Manage.* **29**(2), 18 (2021)
2. Sezer, S., Scott-Hayward, S., Chouhan, P.K., Fraser, B., Lake, D., Finnegan, J., Viljoen, N., Miller, M., Rao, N.: Are we ready for SDN? implementation challenges for software-defined networks. *IEEE Commun. Mag.* **51**(7), 36–43 (2013)
3. Coronado, E., Thomas, A., Riggio, R.: Adaptive ML-based frame length optimisation in enterprise SD-WLANS. *J. Netw. Syst. Manage.* **28**(4), 850–881 (2020)
4. Huo, L., Jiang, D., Qi, S., Miao, L.: A blockchain-based security traffic measurement approach to software defined networking. *Mob. Netw. App.* **26**, 586–596 (2021)
5. Wang, Y., Jiang, D., Huo, L., Zhao, Y.: A new traffic prediction algorithm to software defined networking. *Mob. Netw. App.* **26**, 716–725 (2021)
6. Keshari, S.K., Kansal, V., Kumar, S.: A systematic review of quality of services (QoS) in software defined networking (SDN). *Wireless Pers. Commun.* **116**(3), 2593–2614 (2021)
7. Bencheikh Lehocine, M., Belhadeh, H.: Preprocessing-based approach for prompt intrusion detection in SDN networks. *J. Netw. Syst. Manage.* **32**(4), 79 (2024)

8. Harris, N., Jr., Khorsandroo, S.: Ddt: a reinforcement learning approach to dynamic flow timeout assignment in software defined networks. *J. Netw. Syst. Manage.* **32**(2), 35 (2024)
9. Hassen, H., Meherzi, S., Jemaa, Z.B.: Improved exploration strategy for q-learning based multipath routing in SDN networks. *J. Netw. Syst. Manage.* **32**(2), 25 (2024)
10. Raikar, M.M., Meena, S., Mulla, M.M., Shetti, N.S., Karanandi, M.: Data traffic classification in software defined networks (SDN) using supervised-learning. *Procedia Comput. Sci.* **171**, 2750–2759 (2020)
11. Yang, L., Song, Y., Gao, S., Hu, A., Xiao, B.: Griffin: real-time network intrusion detection system via ensemble of autoencoder in SDN. *IEEE Trans. Netw. Serv. Manage.* **19**(3), 2269–2281 (2022). <https://doi.org/10.1109/TNSM.2022.3175710>
12. Zhang, J., Guo, H., Liu, J.: Adaptive task offloading in vehicular edge computing networks: a reinforcement learning based scheme. *Mob. Netw. App.* **25**, 1736–1745 (2020)
13. Moustafa, S.S., Mohamed, G.-E.A., Elhadidy, M.S., Abdalzaher, M.S.: Machine learning regression implementation for high-frequency seismic wave attenuation estimation in the Aswan reservoir area, Egypt. *Environ. Earth Sci.* **82**(12), 307 (2023)
14. Abdalzaher, M.S., Soliman, M.S., El-Hady, S.M.: Seismic intensity estimation for earthquake early warning using optimized machine learning model. *IEEE Trans. Geosci. Remote Sens.* (2023)
15. Namasudra, S., Lorenz, P., Ghosh, U.: The new era of computer network by using machine learning. *Mob. Netw. App.* 1–3 (2023)
16. Abdalzaher, M.S., Soliman, M.S., El-Hady, S.M., Benslimane, A., Elwekeil, M.: A deep learning model for earthquake parameters observation in IoT system-based earthquake early warning. *IEEE Internet Things J.* **9**(11), 8412–8424 (2022)
17. Salazar, E., Azurdia-Meza, C.A., Zabala-Blanco, D., Bolufé, S., Soto, I.: Semi-supervised extreme learning machine channel estimator and equalizer for vehicle to vehicle communications. *Electronics* **10**(8), 968 (2021)
18. Le, D.C., Zincir-Heywood, N.: A frontier: dependable, reliable and secure machine learning for network/system management. *J. Netw. Syst. Manage.* **28**(4), 827–849 (2020)
19. Sarker, I.H.: Machine learning: algorithms, real-world applications and research directions. *SN Comput. Sci.* **2**(3), 160 (2021)
20. Dong, X., Yu, Z., Cao, W., Shi, Y., Ma, Q.: A survey on ensemble learning. *Front. Comput. Sci.* **14**, 241–258 (2020)
21. Bakker, J., Ng, B., Seah, W.K.G., Pekar, A.: Traffic classification with machine learning in a live network. In: 2019 IFIP/IEEE symposium on integrated network and service management (IM), pp. 488–493 (2019)
22. Bakhshi, T., Ghita, B.: On internet traffic classification: a two-phased machine learning approach. *J. Comput. Netw. Commun.* **2016**(1), 2048302 (2016)
23. Oliveira, A.T., Martins, B.J.C., Moreno, M.F., Gomes, A.T.A., Ziviani, A., Borges Vieira, A.: SDN-based architecture for providing quality of service to high-performance distributed applications. *Int. J. Netw. Manage.* **31**(5), 2078 (2021)
24. Dias, K.L., Pongelupé, M.A., Caminhas, W.M., Errico, L.: An innovative approach for real-time network traffic classification. *Comput. Netw.* **158**, 143–157 (2019)
25. Salau, A.O., Beyene, M.M.: Software defined networking based network traffic classification using machine learning techniques. *Sci. Rep.* **14**(1), 20060 (2024)
26. Eom, W.-J., Song, Y.-J., Park, C.-H., Kim, J.-K., Kim, G.-H., Cho, Y.-Z.: Network traffic classification using ensemble learning in software-defined networks. In: 2021 international conference on artificial intelligence in information and communication (ICAIC), pp. 089–092. IEEE (2021)
27. Yang, T., Vural, S., Qian, P., Rahulan, Y., Wang, N., Tafazolli, R.: Achieving robust performance for traffic classification using ensemble learning in SDN networks. In: ICC 2021-IEEE international conference on communications, pp. 1–6. IEEE (2021)
28. Amaral, P., Dinis, J., Pinto, P., Bernardo, L., Tavares, J., Mamede, H.S.: Machine learning in software defined networks: Data collection and traffic classification. In: 2016 IEEE 24th international conference on network protocols (ICNP), pp. 1–5. IEEE (2016)
29. Boussaoud, K., Ayache, M., En-Nouary, A.: A multi-agent reinforcement learning approach for congestion control in network based-SDN. In: 2023 10th international conference on future internet of things and cloud (FiCloud), pp. 250–255. IEEE (2023)
30. Lin, S.-C., Akyildiz, I.F., Wang, P., Luo, M.: QoS-aware adaptive routing in multi-layer hierarchical software defined networks: a reinforcement learning approach. In: 2016 IEEE international conference on services computing (SCC), pp. 25–33. IEEE (2016)

31. Khairi, M.H.H., Ariffin, S.H.S., Latiff, N.M.A., Yusof, K.M., Hassan, M.K., Al-Dhief, F.T., Hamdan, M., Khan, S., Hamzah, M.: Detection and classification of conflict flows in SDN using machine learning algorithms. *IEEE Access* **9**, 76024–76037 (2021)
32. Buzzio-García, J., Vergara, J., Rios-Guiral, S., Garzón, C., Gutierrez, S., Botero, J.F., Quiroz-Arroyo, J.L., Perez-Diaz, J.A.: SDNFlow dataset. <https://dx.doi.org/10.21227/40v2-hh58>
33. Blagus, R., Lusa, L.: Smote for high-dimensional class-imbalanced data. *BMC Bioinf.* **14**, 1–16 (2013)
34. Fernández, A., Garcia, S., Herrera, F., Chawla, N.V.: Smote for learning from imbalanced data: progress and challenges, marking the 15-year anniversary. *J. Artif. Intell. Res.* **61**, 863–905 (2018)
35. Serag, R.H., Abdalzaher, M.S., Elsayed, H.A.E.A., Sobh, M., Krichen, M., Salim, M.M.: Machine-learning-based traffic classification in software-defined networks. *Electronics* **13**(6), 1108 (2024)
36. Chandra, M.A., Bedi, S.: Survey on svm and their application in image classification. *Int. J. Inf. Technol.* **13**(5), 1–11 (2021)
37. Perera Jayasuriya Kuranage, M., Piamrat, K., Hamma, S.: Network traffic classification using machine learning for software defined networks. In: Machine learning for networking: second IFIP TC 6 international conference, MLN 2019, Paris, Dec 3–5, 2019, Revised Selected Papers 2, pp. 28–39. Springer (2020)
38. Moradi, M., Ahmadi, M., Nikbazzm, R.: Comparison of machine learning techniques for VNF resource requirements prediction in NFV. *J. Netw. Syst. Manage.* **30**(1), 17 (2022)
39. Cover, T., Hart, P.: Nearest neighbor pattern classification. *IEEE Trans. Inf. Theory* **13**(1), 21–27 (1967)
40. Zhao, Y., Li, Y., Zhang, X., Geng, G., Zhang, W., Sun, Y.: A survey of networking applications applying the software defined networking concept based on machine learning. *IEEE Access* **7**, 95397–95417 (2019)
41. Ozsahin, D.U., Taiwo Mustapha, M., Mubarak, A.S., Said Ameen, Z., Uzun, B.: Impact of feature scaling on machine learning models for the diagnosis of diabetes. In: 2022 international conference on artificial intelligence in everything (AIE), pp. 87–94 (2022). <https://doi.org/10.1109/AIE57029.2022.00024>
42. Arfeen, A., Ul Haq, K., Yasir, S.M.: Application layer classification of internet traffic using ensemble learning models. *Int. J. Netw. Manage.* **31**(4), 2147 (2021)
43. Rashid, M., Kamruzzaman, J., Imam, T., Wibowo, S., Gordon, S.: A tree-based stacking ensemble technique with feature selection for network intrusion detection. *Appl. Intell.* **52**(9), 9768–9781 (2022)
44. Bartlett, P., Freund, Y., Lee, W.S., Schapire, R.E.: Boosting the margin: a new explanation for the effectiveness of voting methods. *Ann. Stat.* **26**(5), 1651–1686 (1998)
45. Graczyk, M., Lasota, T., Trawiński, B., Trawiński, K.: Comparison of bagging, boosting and stacking ensembles applied to real estate appraisal. In: Intelligent information and database systems: second international conference, ACIIDS, Hue City, Vietnam, March 24–26, 2010. Proceedings, Part II 2, pp. 340–350. Springer (2010)
46. Ganganwar, V.: An overview of classification algorithms for imbalanced datasets. *Int. J. Emerg. Technol. Adv. Eng.* **2**(4), 42–47 (2012)
47. Musumeci, F., Fidanci, A.C., Paolucci, F., Cugini, F., Tornatore, M.: Machine-learning-enabled DDoS attacks detection in p4 programmable networks. *J. Netw. Syst. Manage.* **30**, 1–27 (2022)

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Rehab H. Serag received the B.Sc. in Electrical Engineering from Ain Shams University, specializing in Communications, in 2018. She pursued a Master of Science in Electrical Engineering focusing on Computers and Systems Engineering. She was awarded a scholarship from the National Telecommunication Institute (NTI) in the networking track, where she further honed her expertise in network technologies. Currently, she serves as a Teaching Assistant at the Egyptian Russian University and is an HCIA Routing & Switching instructor at Huawei Academy – Egyptian Russian University. Her contributions to networking education have been recognized, as she was ranked among the Top 10 instructors in the Network track in 2021. Her research interests include computer networks, Artificial Intelligence, and communication systems.

Mohamed S. Abdalzaher (Senior Member, IEEE) received the B.Sc. (Hons.) in electronics and communications engineering, in 2008, and the M.Sc. degrees in electronics and communications engineering from Ain Shams University, Cairo, Egypt, in 2012, and the Ph.D. degree from the Electronics and Communications Engineering Department, Egypt-Japan University of Science and Technology, Madinet Borg Al Arab, Egypt, in 2016. From April 2019 to October 2019, Dr. Mohamed joined the Center for Japan-Egypt Cooperation in Science and Technology, Kyushu University, where he was a Postdoctoral Researcher. Currently, he is a Research Fellow with the college of engineering, American University of Sharjah.

Dr. Mohamed was elected in the list of top 2% highly ranked scientists. Dr. Abdalzaher was a recipient of the State Encouragement Award, in 2022. He was also a recipient of the Scientific Excellence Award from NRIAG, in 2023 and 2024. He is an Associate Professor with the Seismology Department, National Research Institute of Astronomy and Geophysics, Cairo. Dr. Abdalzaher is an Associate Editor in IEEE Transactions on Geoscience and Remote Sensing and was a scientific editor and reviewer in numerous scientific journals and a TPC member of many scientific conferences. He organized and participated in various conferences and workshops. His research interests include Artificial Intelligence, Wireless Communications, WSN security, IoT, data communication networks, Earthquake Engineering, and Remote Sensing.

Hussein Abd El Atty Elsayed is currently a professor with the Electronics and Communications Engineering Department, Faculty of Engineering, Ain Shams University, Cairo, Egypt, where he received the B.Sc. and M.Sc. degrees respectively. He got the Ph.D. degree from the Electrical Engineering Department, City University of New York, New York, USA. Dr. Elsayed became an IEEE member in 1991. Since graduated, he has served in different positions and built both practical and theoretical skills in the area of telecommunication networks. His fields of interest are ad hoc networks, Cognitive Radio Network (CRN), Wireless Sensor Networks (WSN), Medium Access Control, routing protocols, Software defined Networks (SDN), Device-to-Device communication (D2D), and mobile systems, where he has a long list of publications. Dr. Elsayed supervised and joined several telecommunication graduation, research, and lab/operational projects including service provider Network Operation Center (NOC).

M. Sobh is a distinguished academic and researcher in computer and systems engineering, currently serving as Dean of the Faculty of Engineering Technology at Elsewedy University and a Professor at the Computer and Systems Department, Ain Shams University, Cairo, Egypt. Since earning his engineering degree from Ain Shams University in 1996, Dr. Sobh has dedicated his career to advancing scientific research, teaching, and academic leadership. His expertise spans numerical methods, intelligent systems, algorithms, control systems, machine learning, and information security, and he has played a key role in shaping engineering education in Egypt. Throughout his academic career, Dr. Sobh has held various positions, progressing from Demonstrator to Full Professor at Ain Shams University. He has designed and taught numerous courses, including Big Data Analytics, Software Engineering, Computer Security, Operating Systems, and Numerical Methods, providing students with both theoretical foundations and practical applications. His research contributions have been widely recognized in peerreviewed journals and international conferences, covering areas such as quantum cryptography, malware detection, software-defined networking, and secure embedded systems. His work has not only enriched the academic community but has also influenced technological advancements in cybersecurity, cloud computing, and digital transformation.

Authors and Affiliations

Rehab H. Serag^{1,2}  · Mohamed S. Abdalzaher^{3,4}  ·
Hussein Abd El Atty Elsayed⁵ · M. Sobh¹

✉ Rehab H. Serag
2000764@eng.asu.edu.eg

✉ Mohamed S. Abdalzaher
msabdalzaher@nriag.sci.eg

Hussein Abd El Atty Elsayed
helsayed@eng.asu.edu.eg

M. Sobh
mohamed.sobh@eng.asu.edu.eg

- ¹ Department of Computer and Systems Engineering, Faculty of Engineering, Ain Shams University, Cairo 11566, Egypt
- ² Telecommunication Engineering Department, Faculty of Engineering, Egyptian Russian University, Badr City 11829, Egypt
- ³ Department of Seismology, National Research Institute of Astronomy and Geophysics, Helwan 11421, Egypt
- ⁴ Department of Electrical Engineering, American University of Sharjah, Sharjah 26666, UAE
- ⁵ Department of Electronics and Communications Engineering, Faculty of Engineering, Ain Shams University, Cairo 11566, Egypt